Volatility Modelling and Regime Classification Methods

An understanding exercise

Eloy Sentana Seguí

September 2025

1 Introduction

Volatility is a key measure of risk in financial markets, central to tasks like pricing derivatives, managing portfolios, and assessing market stability. Since volatility changes over time and often clusters in periods of high or low uncertainty, accurate modeling and forecasting are essential.

Models like GARCH capture these dynamics by linking today's volatility to past shocks and variances. Yet markets also shift between different regimes that standard models may miss. To address this, regime classification methods such as Hidden Markov Models (HMM) and Markov Switching Autoregressive (MSAR) models help identify and adapt to structural changes in volatility.

Together, these approaches provide a more complete framework for understanding and anticipating risk in financial markets.

1.1 Motivation

The purpose of this notebook is simple: to help me learn by doing. I wanted to get hands-on with modern volatility estimation and forecasting methods, especially GARCH models and regime-switching approaches like Hidden Markov Models (HMM) and Markov Switching Autoregressive (MSAR) models. Instead of just reading about them, I wanted to actually implement the models, see how they behave with real financial data, and understand what their results really mean.

Along the way, I've tried not only to reproduce results but also to question them. Why this distribution and not another? Why this algorithm, this test, this assumption? Often, digging into these "whys" has taken me beyond the main task and into the theorems behind the models, the statistical tests used for their underlying assumptions, and the principles of mathematical modeling in finance. Those detours have been just as valuable as the direct results.

This notebook doesn't follow strict academic standards: it doesn't cite everything formally, nor does it present full proofs, since the goal was to explore, to test ideas, and to build my own understanding of how volatility can be modeled and forecasted. Working with real data and algorithms has shown me something important: results in practice rarely match the clean outcomes in research papers. And that gap has been one of the most interesting parts of the journey, pushing me to ask even better questions.

I started from the work of Yunhui Wang, but extended his article significantly. I've added my own implementations, experiments, and reflections, building a kind of learning journal where curiosity and questioning are just as central as coding and results.

1.2 Disclaimer

This notebook has not been supervised by any academic or professional in the field. It is purely a personal learning exercise and should not be taken as financial advice or a definitive guide to volatility modeling. If you spot any errors, have any questions, suggestions, or corrections, please feel free to reach out to esentanasegui@gmail.com

---

2 Data

Throughout this practice, we will use the daily adjusted closing prices of the S&P 500 index (^GSPC) with a 20 year lookback. The data is sourced from Yahoo Finance and can be easily accessed using the `yfinance` library in Python.

Figure 1: SP500 Closing Price History

However, using the raw prices is not very useful for statistical modeling, since prices are non-stationary (they have a trend), and their magnitude changes a lot (specially in long timeframes).

Instead, we take the returns for each day, defined as the percentage change from the previous day's price.

However, it is more common to use log returns, due to certain properties that we are about to see.

2.1 Simple Return vs Log

The simple return for a price series is defined as:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

- R_t: simple return at time t
- P_t: price at time t
- P_{t-1}: price at time $t-1$

The logarithmic return (log return) for a price series is defined as:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

- r_t: log return at time t

Although simple returns are easier to understand, it is crucial to understand how log returns work, and why I used them on this practice.

Here is a good example to understand log returns:

Let's suppose a stock has the following (simple) returns for three days:

$$r_1 = 5\%, \quad r_2 = -3\%, \quad r_3 = 4\%.$$

With simple returns, if we want to know the total return after the three days, we should do:

$$P_3 = P_0(1 + r_1)(1 + r_2)(1 + r_3).$$

If $P_0 = 100$, then:

$$P_1 = 100 \times 1.05 = 105,$$
$$P_2 = 105 \times 0.97 = 101.85,$$
$$P_3 = 101.85 \times 1.04 = 105.92.$$

The total simple return is:

$$\frac{P_3 - P_0}{P_0} = \frac{105.92 - 100}{100} = 5.92\%.$$

And it is very important to note that we cannot simply add $5\%-3\%+4\% = 6\%$ because the compounding doesn't allow for addition.

However, with log returns:

$$R_1 = \ln(1.05) = 0.048, \quad R_2 = \ln(0.97) = -0.0304, \quad R_3 = \ln(1.04) = 0.0392.$$

Now, the cumulative log return is just the sum:

$$R_{\text{total}} = R_1 + R_2 + R_3 = 0.048 - 0.0304 + 0.039 = 0.0575.$$

Very important: It is the LOG cumulative return. The corresponding price is:

$$P_3 = P_0\, e^{R_{\text{total}}} = 100 \times e^{0.0575} = 105.92.$$

Which is exactly the compounded simple return, but here the cumulative return was obtained by simply *adding* the log returns.

Another interesting property of taking the log returns is that we are downplaying the big positive returns, and exaggerating the big negative returns. How? This can be seen with the logarithm plot. The effects can bee seen with Figure 4.

Another reason why we use log-returns is for the sake of convenience. In many other models like Black-Scholes for options pricing, normal distribution of the log-returns is assumed, which implies that prices follow a log-normal distribution.

As shown above (Figure 2), prices usually follow a lognormal distribution (not entirely, but it's the best option we have!) per Gregory's (very well explained) post. I have plotted the distribution of prices and then got the best fitting log-normal distribution. We know that prices cannot go lower than 0, and (in theory), they can go up to +inf. So the support of the lognormal distribution and the prices matches.

Additionally, for small daily changes i.e. returns of +-3% (as is the case almost always), simple returns and log returns are almost identical. However, on the long run, log returns are easier to operate with, since for a chain of returns r_i, the total return is given by the product of $(1 + r_i)$ for simple returns, while for log returns, it is simply the sum of the individual log returns. This property simplifies calculations over long periods of time (as it is the case for stock analysis often).

We can now see (Figure 3) the chart of the log returns, distributed accross time. Its mean is very close to 0 (in fact it's 0.0003), and we can see that there are some spikes of high volatility (mostly on crisis and negative events, like COVID, 2008 crisis, etc).

However, to better appreciate the difference between simple returns and log returns, let's plot them (Figure 4):

We can see that they are generally very close, however, we can note the difference better when the return is very high or very low:

1. When returns are very high, the log return is lower than the simple return.
2. When returns are very low (i.e. very negative), the log return is lower (because of the "exxageration")

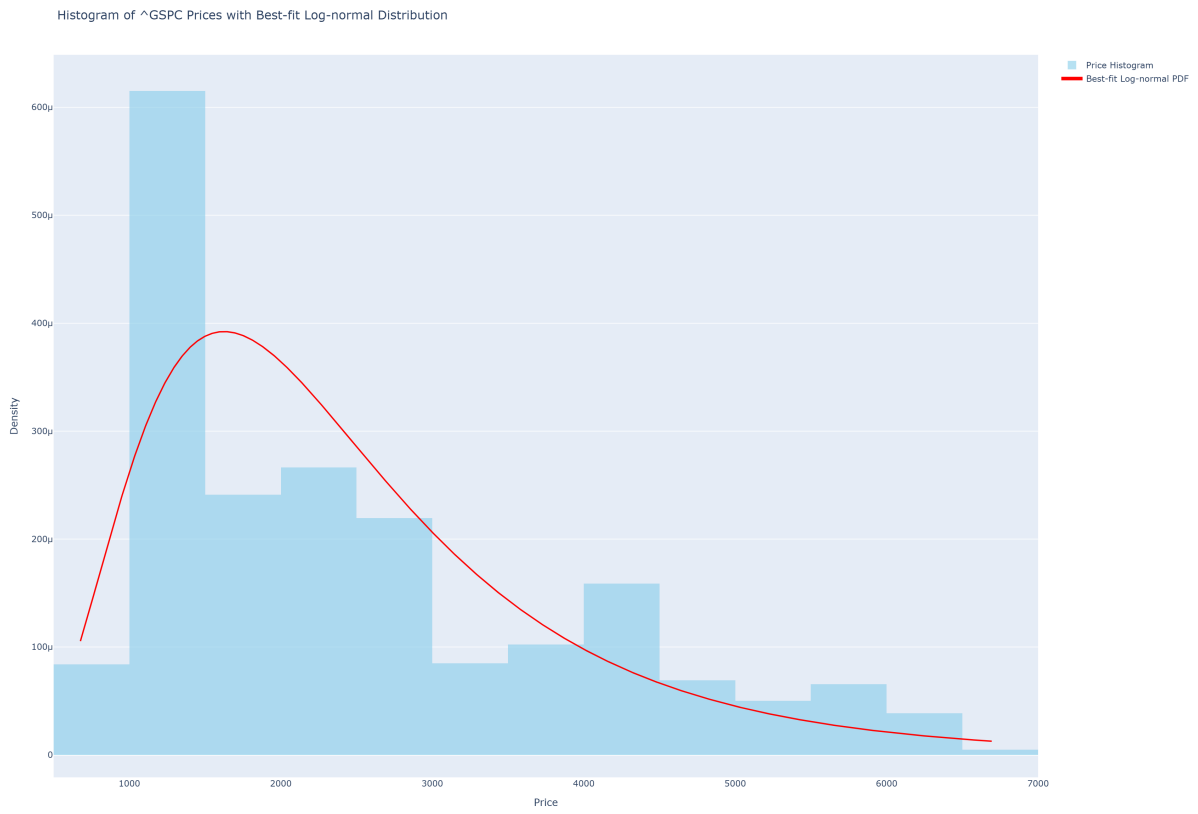

Figure 2: Log-Normal Prices Distribution

Fitted log-normal mean: 2520.10 - Fitted log-normal variance: 2156959.28

2.2 The distribution of the returns

We know what distribution do prices follow (lognormal). But what distribution do returns follow? We are about to see it:

From Figure 5 results, we can interpret the following:

- The mean daily return is slightly positive, i.e. on average the returns tend to be slighly positive.
- The standard deviation is relatively high, i.e. significant day-to-day volatility.
- The negative skewness means that large negative returns are more frequent than large positive returns.
- The high kurtosis (for a normal distribution it's 3) indicates fat tails in the return distribution, meaning extreme returns (both positive and negative) occur more often than would be expected under a normal distribution.
- The Jarque-Bera test strongly rejects the null hypothesis of normality (p-value == 0), confirming that the return distribution is not normal.

But... aren't log returns supposed to make the distribution more normal? Why is the skewness even more negative when taking the log returns? This is because the simple returns already have negative skewness, and since the log returns compress the upside and stretch the downside, the skewness becomes even more negative.

So in this case, the simple returns distribution is (slightly) closer to being a normal distribution (in all aspects except the mean: std dev, skewness, kurtosis, and JB statistic value). This is actually surprising!

However, both returns have very heavy tails, very high kurtosis, and non-zero skewness. I.e. they are not normal (as indicated by the JB test). And this is why we will use GARCH in this practice! Because with GARCH we can use non normal, fat tailed returns!

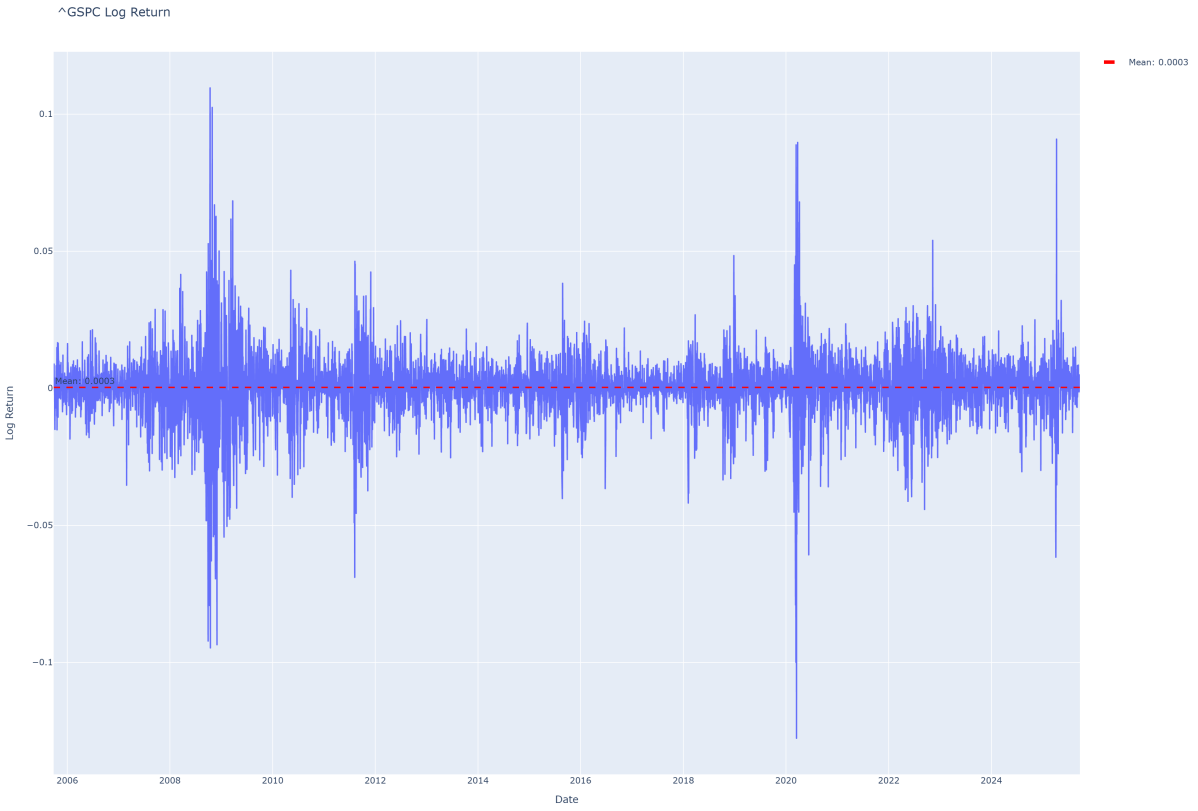

Figure 3: Log Returns over time

2.2.1 Hypothesis testing for mean return and distribution normality

Even though we have already tested for normality of the log returns (and obtained that they are far from normal), let's also test it with some of the arch library built-in tests.

First we will test whether the mean differs significnatly from 0. After running `t_stat, p = stats.ttest_1samp(asset popmean=0, alternative='two-sided')`, we got `p = 0.0503628`. Since our null hypothesis for the mean was that the mean of the return series is 0, if we use an alpha of 0.05, we get that we cannot reject the null hypothesis (i.e. we have no significant statistical proof that the mean differs from 0).

Let's now test for normality: `k2, p = stats.normaltest(return)`.

And we got `p = 3.68834e-258`, so we reject the null hypothesis that the distribution of the log returns was normal.

3 Volatility Modelling with GARCH

The GARCH (Generalized Autoregressive Conditional Heteroskedasticity) model is used to estimate and forecast the volatility of time series, such as financial returns. It is an "advanced" version of the ARCH model introduced by Robert Engle (1982)

3.1 Introduction to the GARCH model

The basic idea is that the variance (volatility) at time t depends on past squared returns (shocks) as well as past variances. This means that volatility tends to cluster around periods of high volatility.

But what do the acronym mean?

1. Generalized: Extends the simpler ARCH model by including past variances as explanatory variables, not just past shocks.

2. Autoregressive: The current volatility depends on its own past values (as in any AR models).

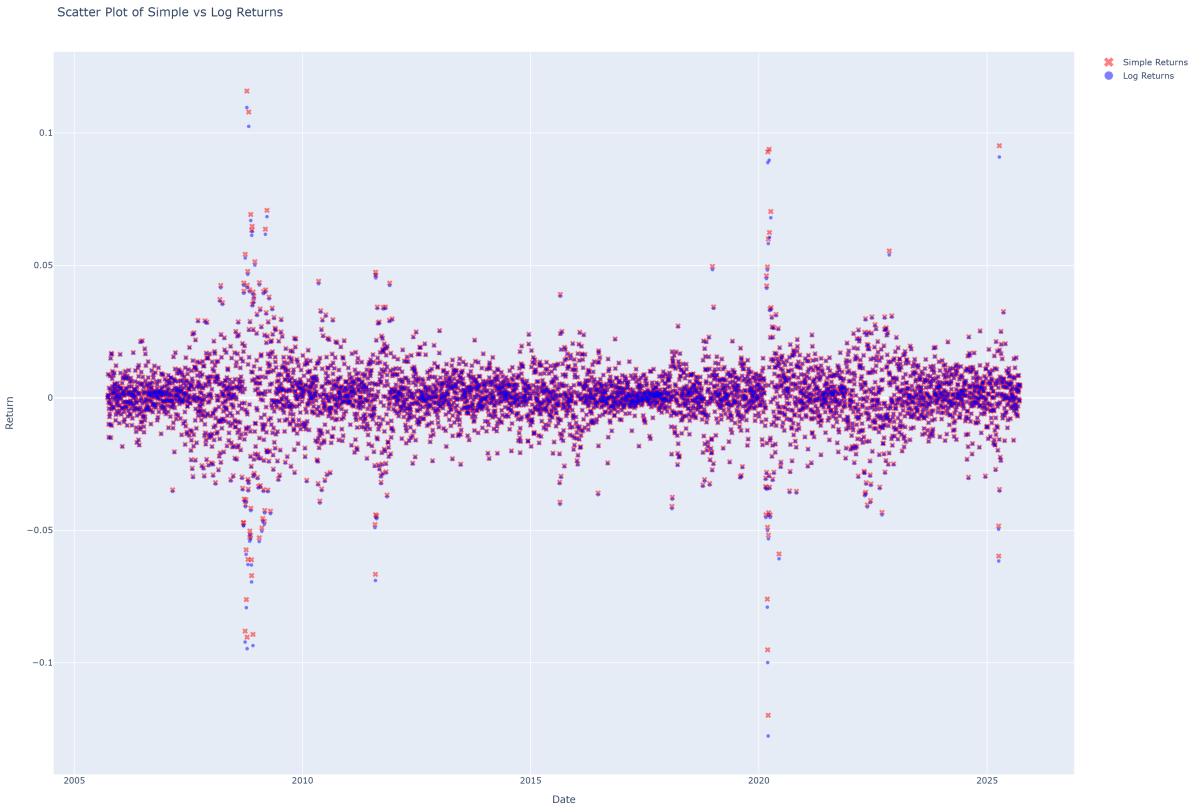

Figure 4: Simple vs Log Returns Scatter Plot

3. Conditional: The variance is estimated with past information (i.e., it is not constant, but conditional on history).

4. Heteroskedasticity: The variance of the error term changes over time (we will return to this later).

The general GARCH(p, q) formula is:

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2$$

Where

ω	constant (baseline volatility)
σ_t^2	conditional variance at time t
α_i, β_j	parameters
ϵ_{t-i}^2	past squared shocks (errors)
σ_{t-j}^2	past variances

ω ensures that the variance never collapses to zero
But... what is the shock ϵ_{t-i}^2? Mathematically, returns are modeled as:

$$r_t = \mu_t + \varepsilon_t$$

Note that ϵ_t and ε_t are used interchangeably (they are just variations of epsilon)
That is, for every return, we try to predict it, but there is always some error (sometimes small, sometimes large) in our guess. That error is the shock.

And how do we get the expected return μ_t? The answer is: it depends. Sometimes a constant is used; other times it is modeled with an ARMA process. In the case of a constant, it is obtained by "training" the model on past data. In some cases it is even set to 0 (in this case, we saw earlier that the mean was not significantly different from 0), because the main focus is volatility.

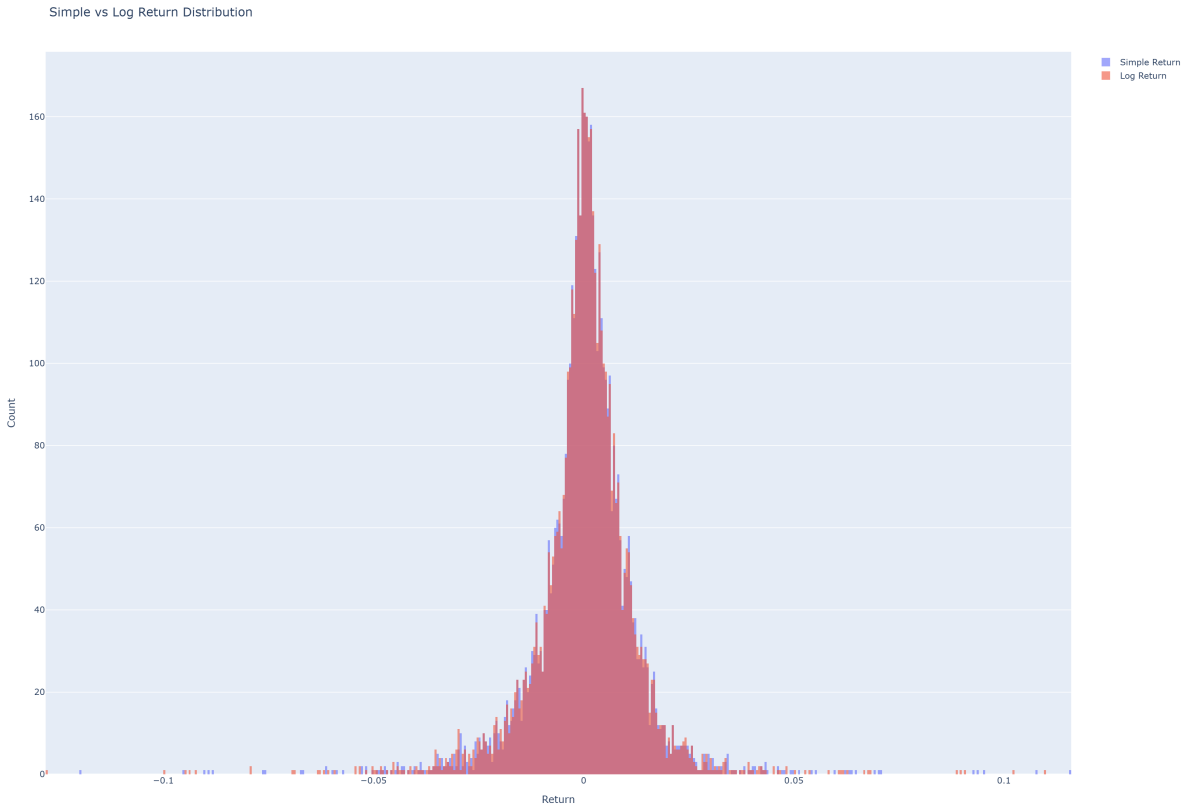

Figure 5: Distribution of Simple vs Log Returns

	Simple Return	**Log Return**
Mean (%)	0.0415	0.0339
Std Dev (%)	1.2268	1.2292
Skewness	-0.2053	-0.4755
Kurtosis	12.7441	12.9136
JB Stat	33993.5461	35057.1092
JB p-value	0.0000	0.0000

Jarque-Bera Test for both distributions:
Simple Return: Not normal (reject H0 at 5% significance level)
Log Return: Not normal (reject H0 at 5% significance level)

The shock is modeled as:

$$\varepsilon_t = \sigma_t z_t$$

where

σ_t conditional volatility at time t (changes over time, modeled by GARCH)
z_t a random variable with mean 0 and variance 1 (often assumed i.i.d. normal)

And why do we square the shock in the GARCH formula? Because volatility must always be non-negative, and we care about its magnitude, not its direction.

With this information, we can better understand the intuition behind the GARCH model: if yesterday's volatility was high, today's volatility is also likely to be high.

Luckily for us, the most widespread version of GARCH is the GARCH(1,1) model (see here):

$$\sigma_t^2 = \omega + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

- Here, today's volatility depends on yesterday's squared shock and yesterday's volatility.

But there is an important property of GARCH: Covariance stationaryness
For this property, we need:

$$\sum_{i=1}^{q} \alpha_i + \sum_{j=1}^{p} \beta_j < 1$$

that ensures that the shocks eventually "die out," and volatility returns to its long-run mean. If

$$\sum_{i=1}^{q} \alpha_i + \sum_{j=1}^{p} \beta_j = 1$$

it is called IGARCH (integrated), where volatility persists at any horizon. If

$$\sum_{i=1}^{q} \alpha_i + \sum_{j=1}^{p} \beta_j > 1$$

volatility explodes (i.e., it increases without bound, which is not the case for financial markets).

And what about the "Heteroskedasticity"? It simply means that the variance of the shock (error) values is not constant across time. Let's visualize this by first plotting the shock values over time (Figure 6), and then seeing their distributions: We will split the data in 4 periods, and see how the distribution of shocks changes accross time.

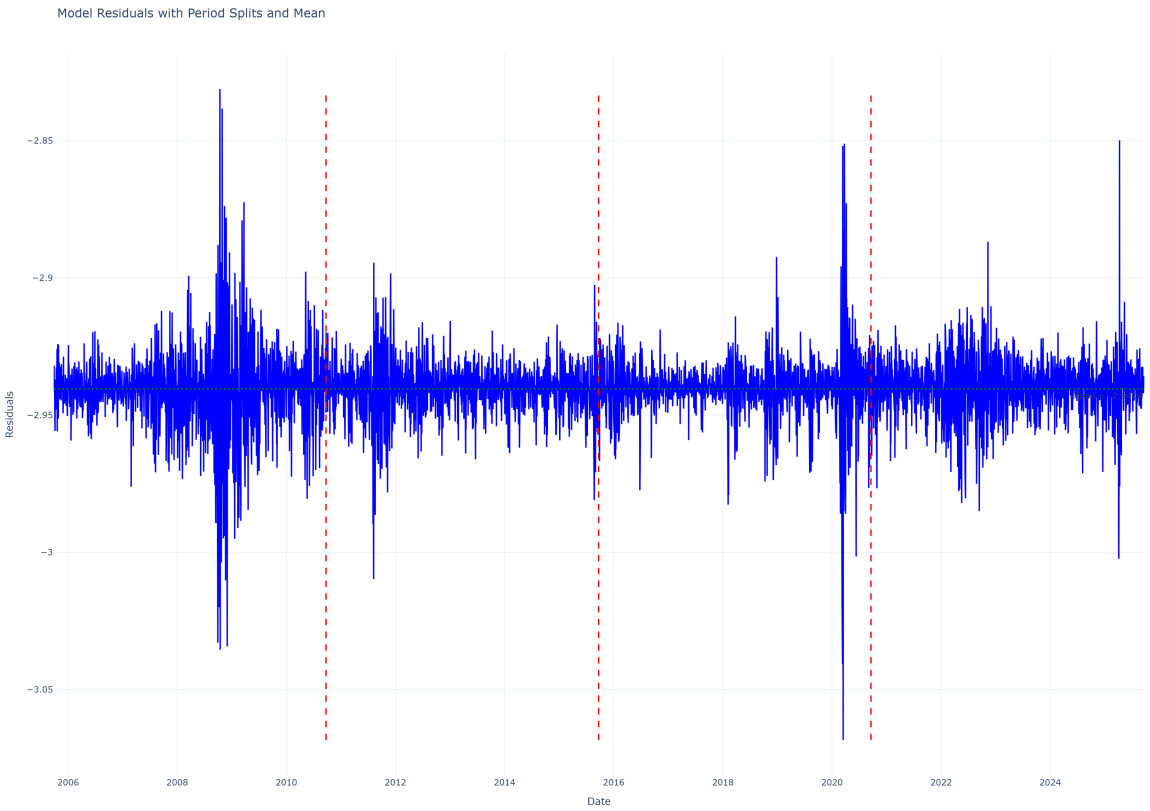

Figure 6: Residuals across time

Here we can see the residuals (i.e. the shocks). if we remember, a return is defined as $r_t = \mu_t + \varepsilon_t$
where μ_t is the expected return (which we set as a constant, found by the model training), and ε_t is the shock (error) at time t. So this graph plots the "surprise" we got each day I.e. actual return - expected return

However, since the variance is hard to see like this, let's get it numerically (Table 1):

We can see (Figure 7) that they are quite different across time! (variance of the first term is 2.5x the variance of the second term). This is mainly because some periods are calmer than others (i.e. less

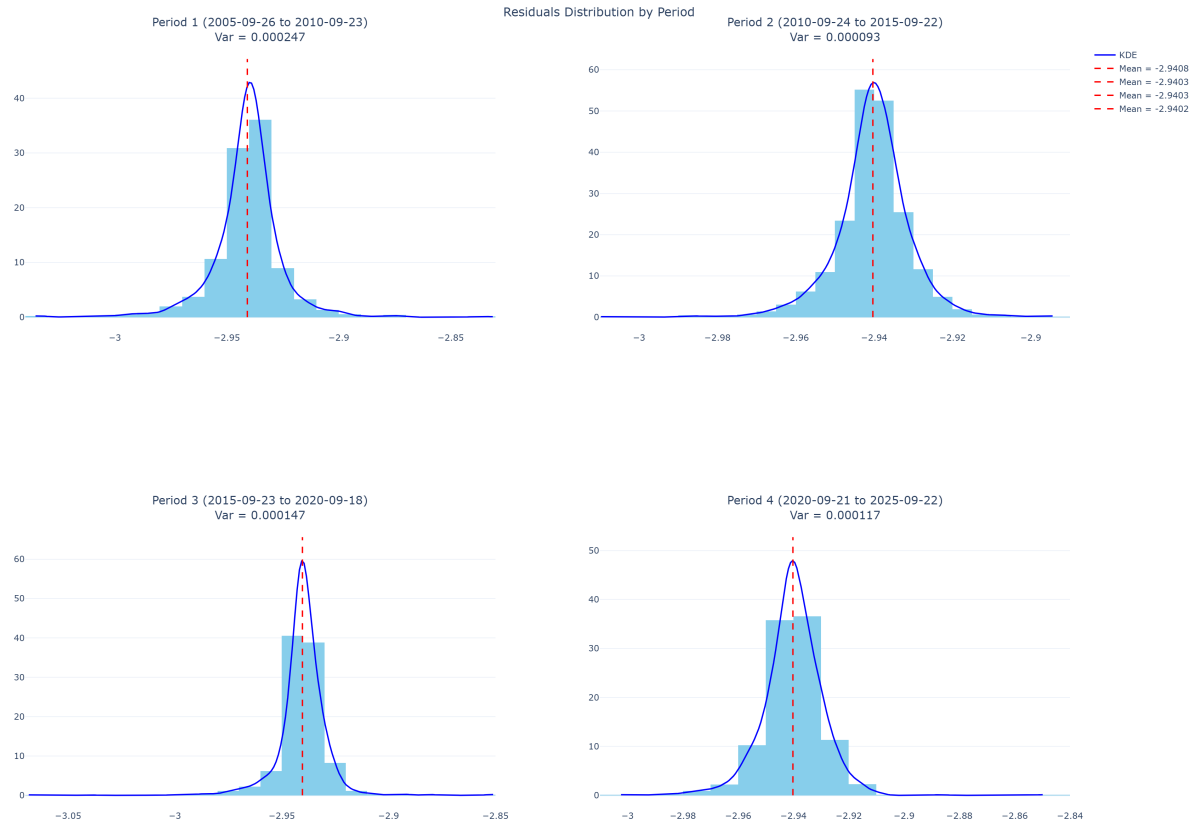

Figure 7: Distribution of shocks over the 4 periods

Period	Start	End	Variance
Period 1	2005-09-26	2010-09-23	0.000247
Period 2	2010-09-24	2015-09-22	0.000093
Period 3	2015-09-23	2020-09-18	0.000147
Period 4	2020-09-21	2025-09-22	0.000117

Table 1: Variance across different periods

big surprises). The KDE plots also show how the distribution of shocks changes across time, with some periods having fatter tails (the KDE is just a smoothed line that takes the values of the histogram, does "mini" distributions and adds them up to get a smooth line).

So we have visually checked that the variance is not constant. But let's now do it analytically, with Robert Engle's Lagrange Multiplier method: it is a test developed by the creator of the ARCH model. It basically tells us that if we have a regressive model $y_t = X_t * \beta + \epsilon_t$ (where X_t is the vector of regressors of the trained garch model we did earlier, and beta the paremeter vector to optimize), then we formulate the following hypothesis:

- Null hypothesis (H0): No ARCH effects (constant variance)
- Alternative hypothesis : We have heteroskedasticity (i.e. changing variance)

So we first estimate the model with Ordinary Least Squares (i.e. basically we try to find a line/hyperplane that is closest to the data points, i.e. fitting) so that we have a relationship between the dependent variable y and the independent variable(s) x1, x2, etc. So it is essentially a minimization problem, where we try to minimize the sum of squared shocks (errors).

After getting this regression, we square the residuals (shocks), and then we build the following regression:

$$\hat{\epsilon}_t^2 = \alpha_0 + \alpha_1 \hat{\epsilon}_{t-1}^2 + \alpha_2 \hat{\epsilon}_{t-2}^2 + ... + \alpha_p \hat{\epsilon}_{t-p}^2 + u_t$$

The $\hat{\epsilon}_t^2$ are the squared residuals (shocks) we got from the first regression, and we regress them on their own lags (i.e. past values of themselves). The number of lags p is a parameter we choose!

Then we estimate again with OLS the alpha parameters, and if these params are jointly significant, i.e. if they are statistically significantly different from 0, then we reject the null hypothesis of homoskedasticity (constant variance), and we can conclude that there is heteroskedasticity (changing variance).

Once we have that reggression, we get R^2 = 1 - SumSquaredResiduals/SumSquaredTotal and then we compute the LM statistic:

$$LM = n \cdot R^2$$

, where n is the lenght of the return series. Therefore, if there are no ARCH effects, the $\hat{\epsilon}_t^2$ should not be explained by its own lags, and R^2 should be close to 0. If there are ARCH effects, then the lagged squared errors should explain a significant portion of the variance in the current squared errors, so the R^2 value must be higher.

$LM = n \cdot R^2$ follows a Chi-squared distribution (proved by Engle) with q degrees of freedom (where q is the number of lags we used in the regression). So we can get the p-value from this distribution, and if it is lower than 0.05 (i.e. 5% significance level), then we reject the null hypothesis of homoskedasticity (constant variance), and we can conclude that there is heteroskedasticity (changing variance).

The result of doing

```
test_stat, p_value, _, _ = statsmodels.stats.diagnostic.het_arch(res.resid)
```

is

```
ARCH test statistic: 108.75357748583721
p-value: 9.515470479583062e-19
Evidence of heteroskedasticity: the variance of the errors changes over time.
```

The ARCH test statistic is the LM = n * R^2 value. The p-value is very small (below our significance level) so we reject the null hypothesis, and we can determine that there is heteroskedasticity (changing variance)!

With these plots (and the ARCH variance test), we can see that the variance is not constant accross the 4 periods (but this extends to any number of periods!). We see that the variances differ significantly, and it makes sense logically: On the first period, the 2008 crisis created more volatility (i.e. higher variance), on the second period, large shocks were rare (only the 2010 European debt crisis stands out, but in general it was a "calm" period). On the thrid period, the COVID-19 crisis increased volatility, and on the 4th period only the tariffs caused some shocks.

With this (both analytical and visual) analysis, we have explained why a model that takes into account the Heteroskedasticity of variances is important.

So, going back to the GARCH model, what about the α and β parameters? They are not set manually. During the "training", they are estimated using maximum likelihood estimation (MLE). The model assumes returns follow a conditional variance process:

$$\sigma_t^2 = \omega + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

- ω: baseline variance
- α_1: impact of recent shocks
- β_1: persistence of past variance

MLE finds the parameter values that maximize the likelihood of the observed data, given the model. On this practice we use the arch Python package that does the parameter finding automatically. Thus, α and β are learned from the data to best fit the observed returns.

Let's thus build a model with the log returns (`rt`) and the `arch` Python library.

```
am = arch.univariate.arch_model(rt, x=None,
                                mean='constant', lags=None,
                                vol='Garch', p=1, o=0, q=1,
                                dist='normal', hold_back=None, rescale=True)

volatility_model = am.fit()
volatility_model.summary()
```

and we get:

```
Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable: Close   R-squared: 0.000
Mean Model: Constant Mean   Adj. R-squared: 0.000
Vol Model: GARCH   Log-Likelihood: -6766.28
Distribution: Normal   AIC: 13540.6
Method: Maximum Likelihood   BIC: 13566.6
No. Observations: 5029
Date: Tue, Sep 19 2025   Df Residuals: 5028
Time: 00:43:24   Df Model: 1

Mean Model
------------------------------------------------------------------------------
        coef       std err     t        P>|t|    95.0% Conf. Int.
mu      0.0740     0.0110   6.709   1.964e-11   [0.052, 0.096]

Volatility Model
------------------------------------------------------------------------------
        coef       std err  t       P>|t|       95.0% Conf. Int.
omega   0.0281     0.0055   5.084   3.701e-07   [0.017, 0.039]
alpha1  0.1363     0.0142   9.590   8.842e-22   [0.108, 0.164]
beta1   0.8426     0.0143   58.932  0.000e+00   [0.815, 0.871]
==============================================================================
```

3.1.1 Long-term variance under GARCH(1,1)

Let's now explore a concept that I find very interesting: What happens when we want to make a very far away prediction? Do we get the mean of all the variances filtered by our model? Or is there any other alternative?

Here are the parameters of our fitted GARCH(1,1) model:

Parameter	Estimate
μ	0.073999
ω	0.028126
α_1	0.136257
β_1	0.842551

Table 2: Estimated parameters of the GARCH(1,1) model

Let's break them down:

1. mu: This is the mean (expected return) of the time series

2. omega: This is the constant term in the variance equation (i.e. what makes volatility never go to 0)

3. alpha[1]: It measures how much yesterda's squared shock affects today's volatility

4. beta[1]: It measures how much yesterday's volatility affects today's vol. (higher beta = more persistance = more vol clustering)

Therefore our GARCH(1,1) model is defined by:

$$\sigma_t^2 = \omega + \alpha\,\epsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2$$

And with our values:

$$\sigma_t^2 = 0.028126 \; + \; 0.136257\,\epsilon_{t-1}^2 \; + \; 0.842551\,\sigma_{t-1}^2$$

Note: if this test is rerun in the future, yfinance will provide the last* 20y of data, so the estimated parameters values will not be the same

Long-term variance is the level of variance that the process will converge to if you let time go to infinity (do not mistake it with the omega value!). Indeed, the long term variance is defined with:

$$\sigma_\infty^2 \; = \; \frac{\omega}{1-\alpha-\beta},$$

and since Volatility is the sqrt of the Variance:

$$\sigma_\infty = +\sqrt{\frac{\omega}{1-\alpha-\beta}}.$$

Therefore, if we want a rough estimate of what will volatility do in a long time, we can take σ_∞ as an estimate! In fact, let's compute it wiht `VL = omega / (1 - alpha - beta)` (and interpret the results):

```
Long-term variance under GARCH: 1.33 / (annualized: 21.07)
Long-term volatility under GARCH: 18.29 %
Sample volatility estimate: 19.51 %
```

The long term volatility tells us that our model expects returns to fluctuate about +-18.29% per year around the mean.

Sample volatility estimate: This is the actual vol we had on our dataset (annualized too)

Since the sample volatility long-term GARCH volatility, it means that the model expects the future to be less turbulent than our test period. This tells us that GARCH is more forward looking than our sample estimate. This result makes sense, since our dataset included fairly volatilie periods, and our alpha and beta values indicate high persistence of volatility.

3.1.2 Conditional volatility

Let's now plot the absolute returns and the conditional volatility together (Figure 8). This conditional volatility is the one obtained after estimating the GARCH parameters. The model goes back in time, and estimates the volatility for each day, given the past shocks and volatilities.

**Note that I have plotted them this way to see how reactive the conditional volatility is to the sudden changes in returns, not the magnitude of the spikes. In fact, they are in different scales!*

We can see in Figure that our model follows the actual returns pretty well. As one could expect, it is very correlated.

However, the interesting thing happens when we zoom in (preferrably somewhere with a big spike in returns). We used the COVID-19 volatility spike here (Figure 9):

We can see that the GARCH volatility is slightly delayed. And this makes sense, since as we explained earlier, the GARCH model is **autoregressive**, i.e. it uses its *own past data*, lagged by 1 period (days in this case).

And with this other zoomed plot (Figure 10) we can see the effects of having a high $\alpha + \beta$:

1. We see that the GARCH volatility is very persistent (it decays much slower than the returns).

But why are we using a rolling window for the realized volatility? Because getting the true variance every day is too volatile, so with the rolling window, we can get a better comparison method.

4 Future volatility forecasting with GARCH

We have seen how we can estimate the *past* volatility values *after* estimating its parameters. On this section however, we will focus on estimating *future* volatility, which is more useful for real life applications (where anticipating volatility levels helps hedge risks, or compute the level of exposure to risk a portfolio has, just like with VaR modelling for example).

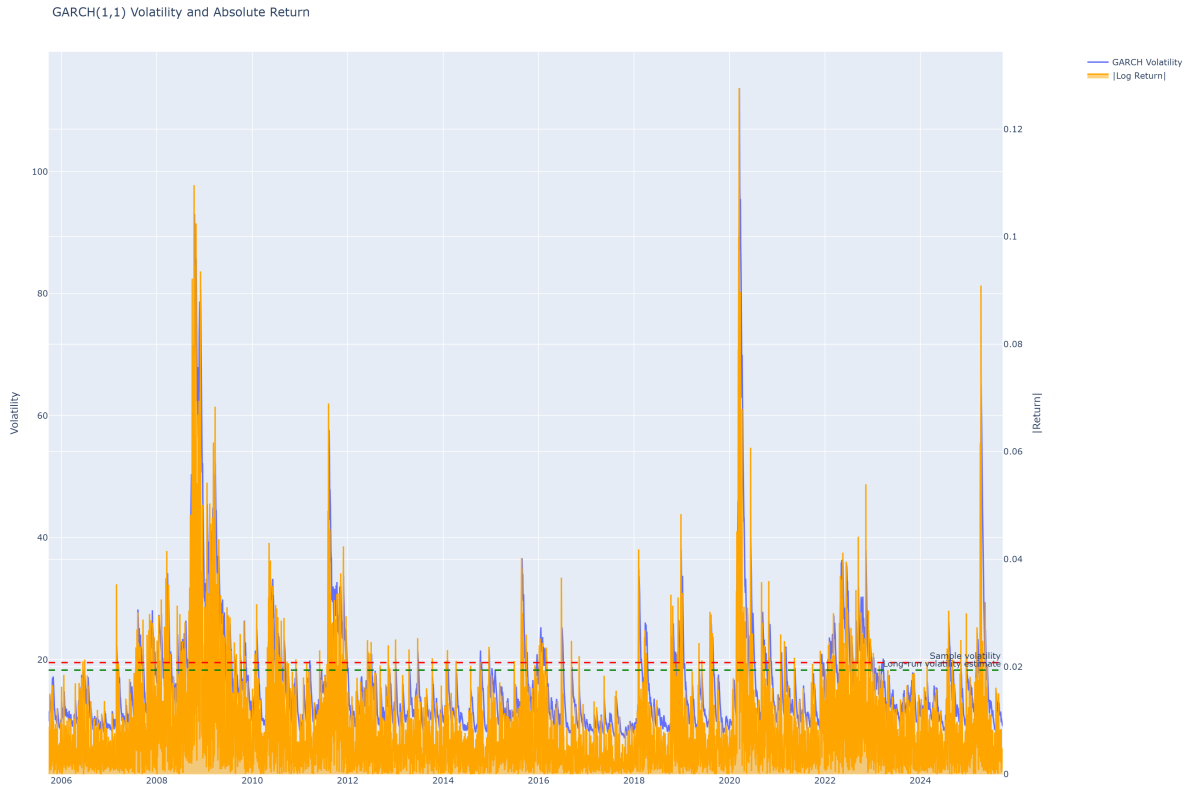

Figure 8: Volatility and Absolute Returns

4.1 Blind volatility prediction

Sometimes we want to estimate the volatility in N months' time without estimating the market prices first (like being blind to the market). For this, we will implement volatility prediction using GARCH. We will train the model again, and for this, we will:

1. Split the data into $X\%$ training and $(100 - X)\%$ testing.
2. "Train" the GARCH model on the training data (i.e. get the parameters).
3. Make predictions for every day of the test period.
4. Evaluate the prediction performance with the test period's realized volatility.

That is, we will predict volatility as if we had **no** information about the market (just the training data) during the test period, and then we will compare it with future test data to see how we did.

But how does the prediction work? By observing the GARCH formula ($\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$) one would wonder: How does the model get the ϵ_{t-1}^2 at each iteration without access to more information?

Let's see it step by step to identify the issue (and solution!):

On the first iteration, we have the very last ϵ_0^2 and σ_0^2 of the fitted model. Therefore we have no issues getting σ_1^2:

$$\sigma_1^2 = \omega + \alpha\,\epsilon_0^2 + \beta\,\sigma_0^2$$

On the second iteration, we want to obtain:

$$\sigma_2^2 = \omega + \alpha\,\epsilon_1^2 + \beta\,\sigma_1^2$$

but we only have σ_1^2, not ϵ_1^2! Instead of panicking, we use the *estimate* of ϵ_1^2 i.e., we take the conditional expectation given today's information $\mathcal{I}_0$:

$$E[\sigma_2^2 \mid \mathcal{I}_0] = \omega + \alpha\,E[\epsilon_1^2 \mid \mathcal{I}_0] + \beta\,E[\sigma_1^2 \mid \mathcal{I}_0].$$

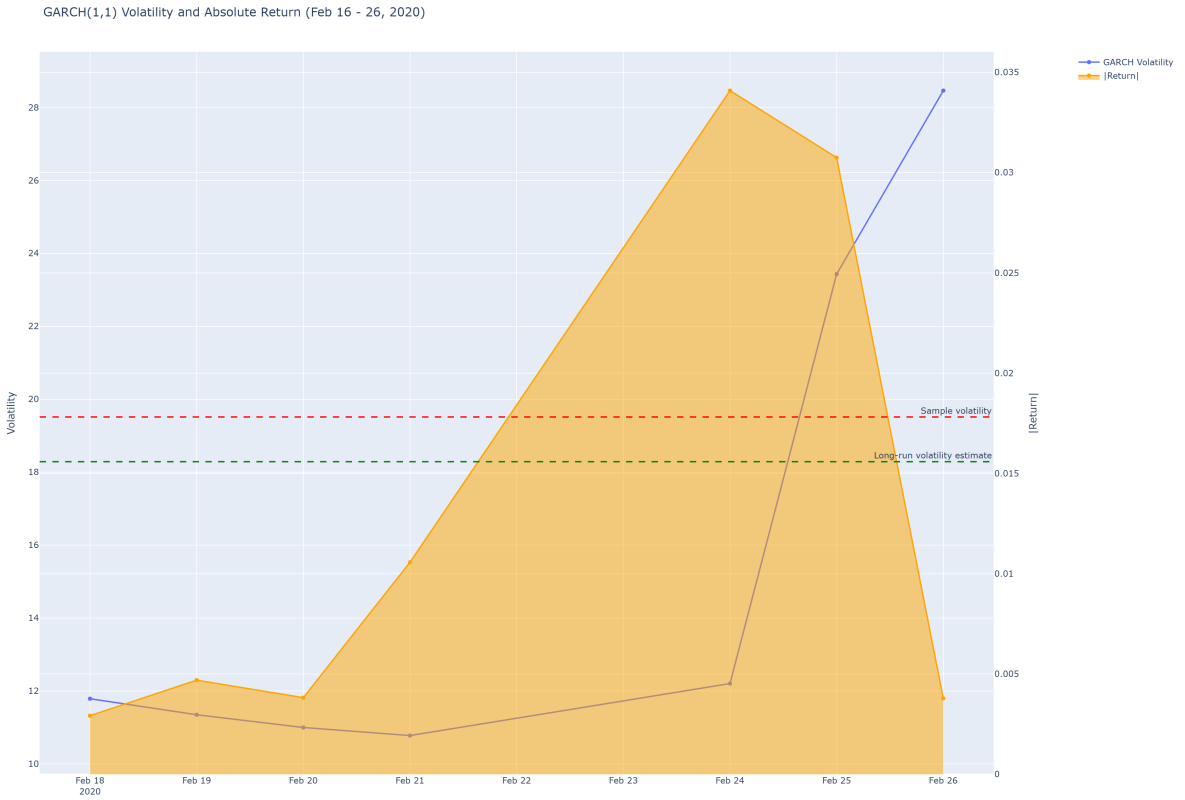

Figure 9: Lagged Conditional Volatility w.r.t abs. Returns

(And note: σ_1^2 is already determined by the first step from $\mathcal{I}_0$, so $E[\sigma_1^2 \mid \mathcal{I}_0] = \sigma_1^2$.)
And how do we get $E[\epsilon_1^2 \mid \mathcal{I}_0]$? As explained before, ϵ_t is defined as:

$$\varepsilon_t = \sigma_t z_t$$

so $\varepsilon_t^2 = \sigma_t^2 z_t^2 \implies E[\varepsilon_t^2 \mid \mathcal{I}_{t-1}] = E[\sigma_t^2 z_t^2 \mid \mathcal{I}_{t-1}]$.
And if we remember, z_t has mean 0 and variance 1. Therefore, with simple probability:

$$\text{Var}(z_t) = E[z_t^2] - (E[z_t])^2 \implies E[z_t^2] = 1.$$

Assuming z_t is independent of $\mathcal{I}_{t-1}$ (as explained before):

$$E[\varepsilon_t^2 \mid \mathcal{I}_{t-1}] = E[\sigma_t^2 \mid \mathcal{I}_{t-1}] \cdot E[z_t^2] = E[\sigma_t^2 \mid \mathcal{I}_{t-1}] \cdot 1 = E[\sigma_t^2 \mid \mathcal{I}_{t-1}].$$

Apply this now to the GARCH recursion for forecasting (with $t = 1$ and conditioning on $\mathcal{I}_0$):

$$E[\sigma_2^2 \mid \mathcal{I}_0] = \omega + \alpha\, E[\epsilon_1^2 \mid \mathcal{I}_0] + \beta\, E[\sigma_1^2 \mid \mathcal{I}_0] = \omega + \alpha\, E[\sigma_1^2 \mid \mathcal{I}_0] + \beta\, E[\sigma_1^2 \mid \mathcal{I}_0] = \omega + (\alpha + \beta) E[\sigma_1^2 \mid \mathcal{I}_0]$$

and since, from before, we know that $E[\sigma_1^2 \mid \mathcal{I}_0] = \sigma_1^2$, we get:

$$E[\sigma_2^2 \mid \mathcal{I}_0] = \omega + (\alpha + \beta) E[\sigma_1^2 \mid \mathcal{I}_0] = \omega + (\alpha + \beta)\sigma_1^2$$

And this pattern continues for all the next iterations. For $t \geq 1$:

$$E\left[\sigma_{t+1}^2 \mid \mathcal{I}_0\right] = \omega + (\alpha + \beta)\, E\left[\sigma_t^2 \mid \mathcal{I}_0\right]$$

This is why, when forecasting volatility, the GARCH prediction converges to its long-term variance, since each new step uses the previous forecasted variance as a proxy for the (unseen) squared shock! And this is the mathematical reason why, without new information, the forecasted volatility "forgets" past shocks and drifts toward the unconditional (long-run) volatility level.

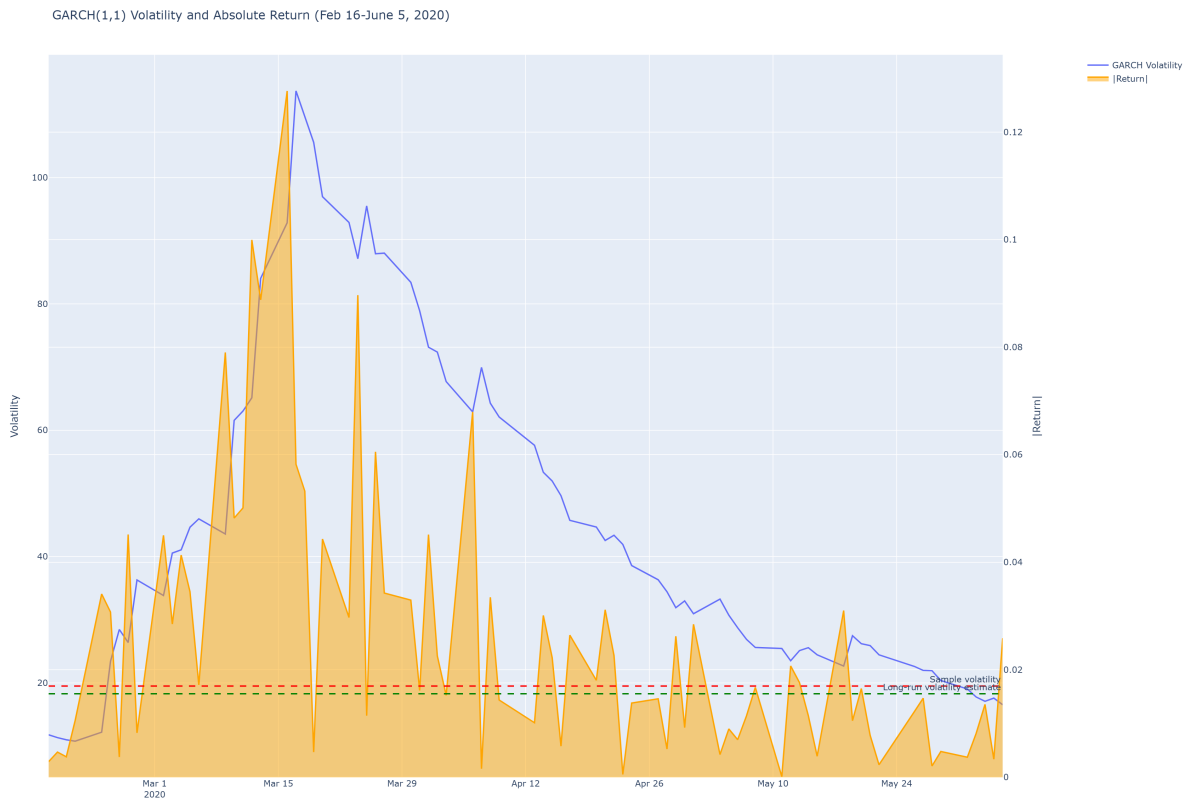

Figure 10: Persistence of volatility

Let's now predict the future volatility of the SP 500. We took 95% of the data to train the model, and 5 to test it. That is, we will predict the volatility of every day for a year.

```
Training data: 4777 observations (2005-09-26 00:00:00-04:00 to 2024-09-18 00:00:00-04:00)
Test data: 252 observations (2024-09-19 00:00:00-04:00 to 2025-09-22 00:00:00-04:00)
```

We first train the model with 95% of the data, and get the following parameters:

Parameter	Estimate
μ	0.072222
ω	0.027085
α_1	0.135431
β_1	0.844168

Table 3: Estimated parameters of the GARCH(1,1) model

We then do `test_predictions = train_fitted.forecast(horizon=len(test_returns), reindex=False)` to get the daily variance predictions, and then we convert it to annualized volatility estimates with `test_vol_predictions = np.sqrt(test_predictions.variance.values[-1, :]) * np.sqrt(252)`.

We use 252 because it is the average number of trading days per year.

```
Prediction Performance Metrics:
----------------------------------------
Mean Squared Error (MSE): 165.3892
Root Mean Squared Error (RMSE): 12.8604
Mean Absolute Error (MAE): 8.6207
Mean Absolute Percentage Error (MAPE): 91.28%
```

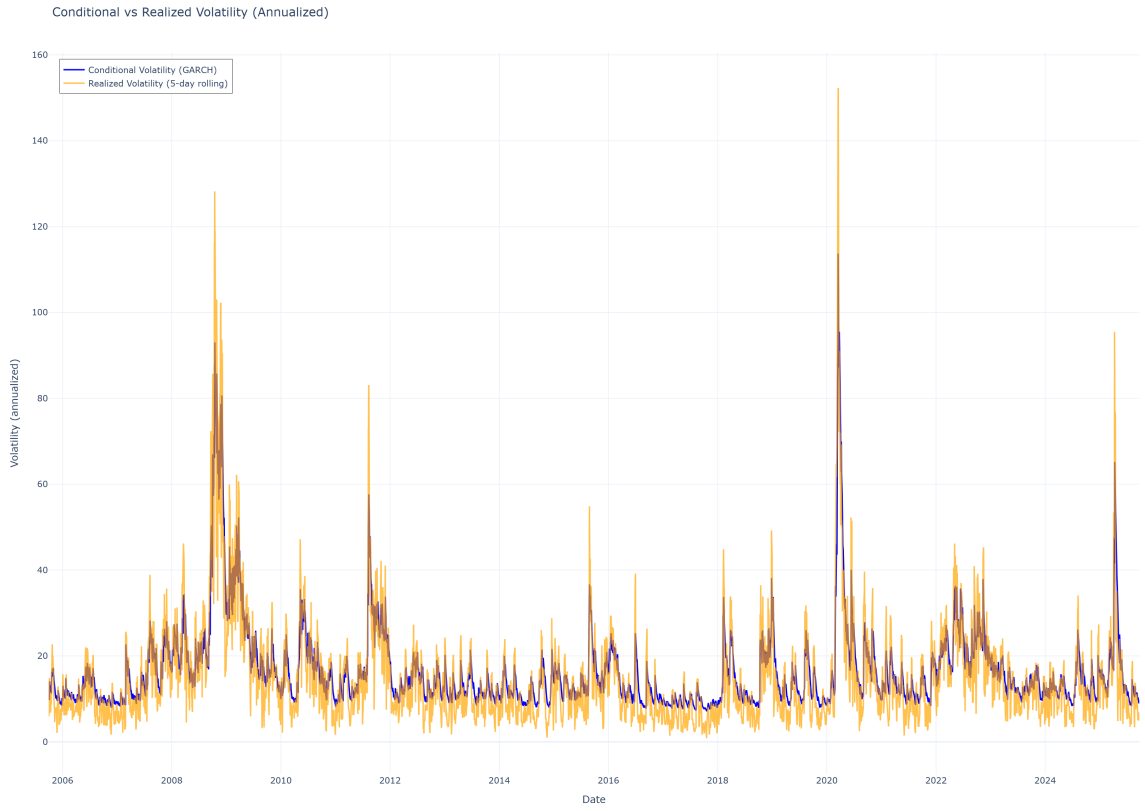

Figure 11: Conditional vs Realized Volatility

```
Correlation: 0.1535

Volatility Comparison:
----------------------------------------
GARCH Long-term Volatility: 18.29
Test Period Average Realized Volatility: 14.32
```

Our prediction is terrible! We see that:

- MSE (165.39): Captures average squared errors; large deviations are heavily penalized. So it's telling us that there were some very large prediction errors.
- RMSE (12.86): This is the forecast error in annualized volatility units.It tells us that the errors are almost as large as the volatilities themselves!
- MAE (8.62): It tells us that our forecasts miss by about 8–9 vol points on average.
- MAPE (91.28%): Relative error; predictions are off by nearly the same magnitude as realized volatility.

The GARCH(1,1) model is overpredicting volatility (long-term GARCH vol of 18.29 vs. test period avg. realized vol 14.32) and failing to track short-term dynamics (low correlation, very high MAPE). This is exactly what we would expect for a simple GARCH(1,1) when forecasting long horizons.

But how does such a bad prediction look like? We are about to find out a very interesting result:

On the first plot (Figure 12), we can see the conditional volatility (annualized) of the fitted model over the train period, and the prediction over the test/forecast period (5%) (without test data). We can see that the prediction is a smooth curve, contrasting with the spiky conditional volatility.

But it's on Figure 13 where we can see some interesting things:

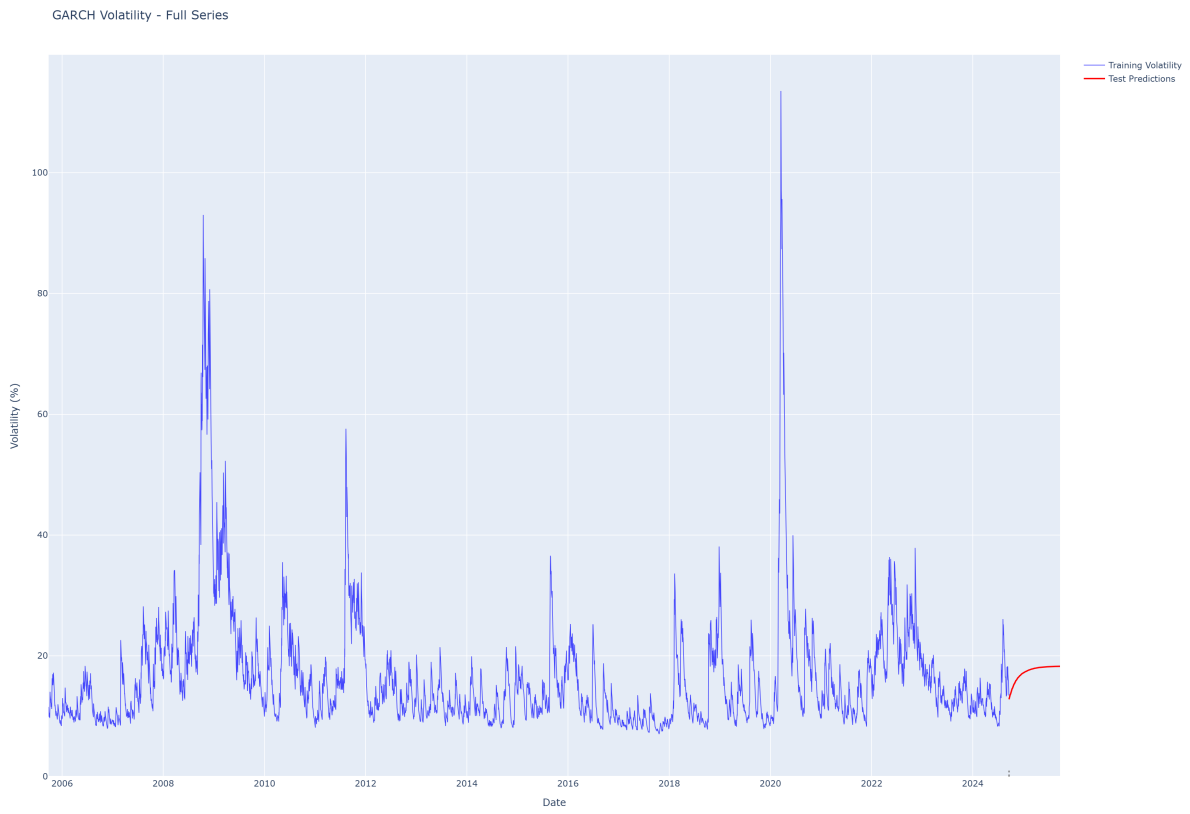

Figure 12: Trained Model Conditional Vol. and 1y Predicted Vol.

1. There is a big difference between the test avg. realized volatility and the model's long-term volatility. This indicates that we trained the model on more volatile data than the one for testing, since the model is constantly overpredicting volatility.

2. Without observations of the market, the prediction converges to the long-term volatility, given by

$$\sigma^2 = \frac{\omega}{1 - \alpha_1 - \beta_1} = \frac{0.027085}{1 - 0.135431 - 0.844168} \approx 1.32$$

$$\sigma = \sqrt{1.32} \approx 1.15 \quad \text{(convert variance into volatility)}$$

$$\sigma_{\text{annual}} = \sigma \times \sqrt{252} = 1.15 \times \sqrt{252} \approx 18.29 \quad \text{(annualized long-term volatility)}$$

4.2 Daily Retrain GARCH model

On the initial model training (see Volatility Modelling with GARCH section), once we trained the model, and got the parameters, we could use it to estimate the past volatility values, by doing "filtering". We can estimate the volatility for the next days by iteratively using the GARCH formula, with the volatility obtained on the previous day and the realized shock that we would observe on the market every day.

While that approach is valid, it has a flaw: The parameters obtained by doing an initial "training" of the model are never again updated. On the short term, this is not a problem, but if we kept the same parameters for a long time, our model would be applying to today's forecasting a baseline volatility and characteristics (alpha, beta, persistence) that was identified on the past.

In this section, I implemented a more realistic forecasting approach where:

1. We start with an initial training window (95% of the available data).

2. Make a 1-day ahead inference of σ^2_{t+1} (using σ^2_t and ϵ^2_t).

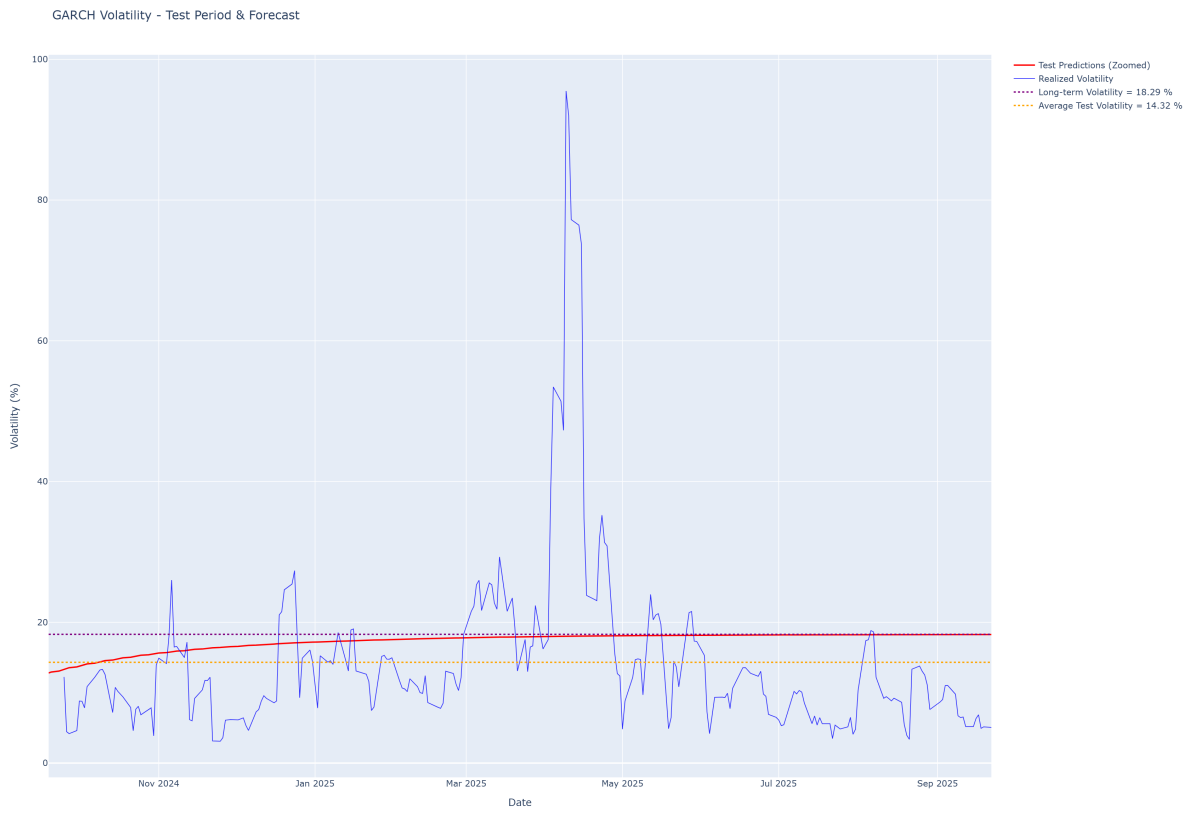

Figure 13: Test period: Prediction vs Realized volatility

3. Retrain the model with the newly observed return.
4. Repeat the process for each subsequent day.

This approach resembles way more to the real-world forecasting where models are continuously updated with new information. However, it is very compute-intensive (1 model fitting per period).

Note that here, "realized volatility" is the absolute daily returns (i.e. we are assuming the mean of returns is 0, since the model used for each day has a different mean, and we showed earlier that the mean is not significantly different from 0.

This plot is very similar to the initial graph we saw when exploring what the GARCH model was. Here we can also see that the the t+1 volatility is one-period lagged with respect to the returns.

But... do parameters change much over time when retraining the model every day? Let's see it:

Table 4: Parameter Evolution Statistics

Parameter	**Mean**	**Std**	**Min**	**Max**	**% Change**
ω	0.028021	0.000890	0.026557	0.030450	6.85
α_1	0.140069	0.003949	0.135192	0.149548	8.23
β_1	0.840830	0.003281	0.832814	0.845823	-1.17
Persistence	0.980899	0.001047	0.978528	0.983789	0.14
Long-term vol.	1.212466	0.039839	1.176305	1.320833	7.25

These results highlight several interesting things:

1. The parameters α and β appear to be (roughly) inversely correlated.
2. The long-term volatility reacts quickly to shocks, largely driven by the α component.
3. Persistence remains relatively stable throughout the training period.
4. Over time, the model increasingly emphasizes the sequence of shocks (progression-wise) rather than the magnitude of past volatility values.

Figure 14: Daily predictions

4.3 Train and Run approach

We mentioned earlier, that after obtaining the parameters of the model, we could use the formula and the realized shocks of every passing day to calculate the conditional volatility of that day (and use it to compute the one for the next day).

The procedure is as follows:

1. Train the GARCH model on the entire training dataset (a fixed percentage of the full dataset).
2. Extract the model parameters after training.
3. Use these parameters to make future predictions on the test dataset without retraining, applying the standard recursion formula.

With this setup, we aim to evaluate how the model performs, particularly in comparison with the previous approach (which was retrained daily).

We refer to the once-trained model as the *Outdated* model.

The two models under comparison (Figure 16) are:

1. A model trained on the first 50% of the data and never again refitted.
2. A model initially trained on the first 50% of the data, but subsequently retrained daily on the remaining 50%, so that the final fitted model incorporated all available data (excluding the last period).

We can see (Figure 17) that both models are almost identical. However, we see that with time, the daily retrained model is more reactive to shocks, while having less persistence overall to volatility.

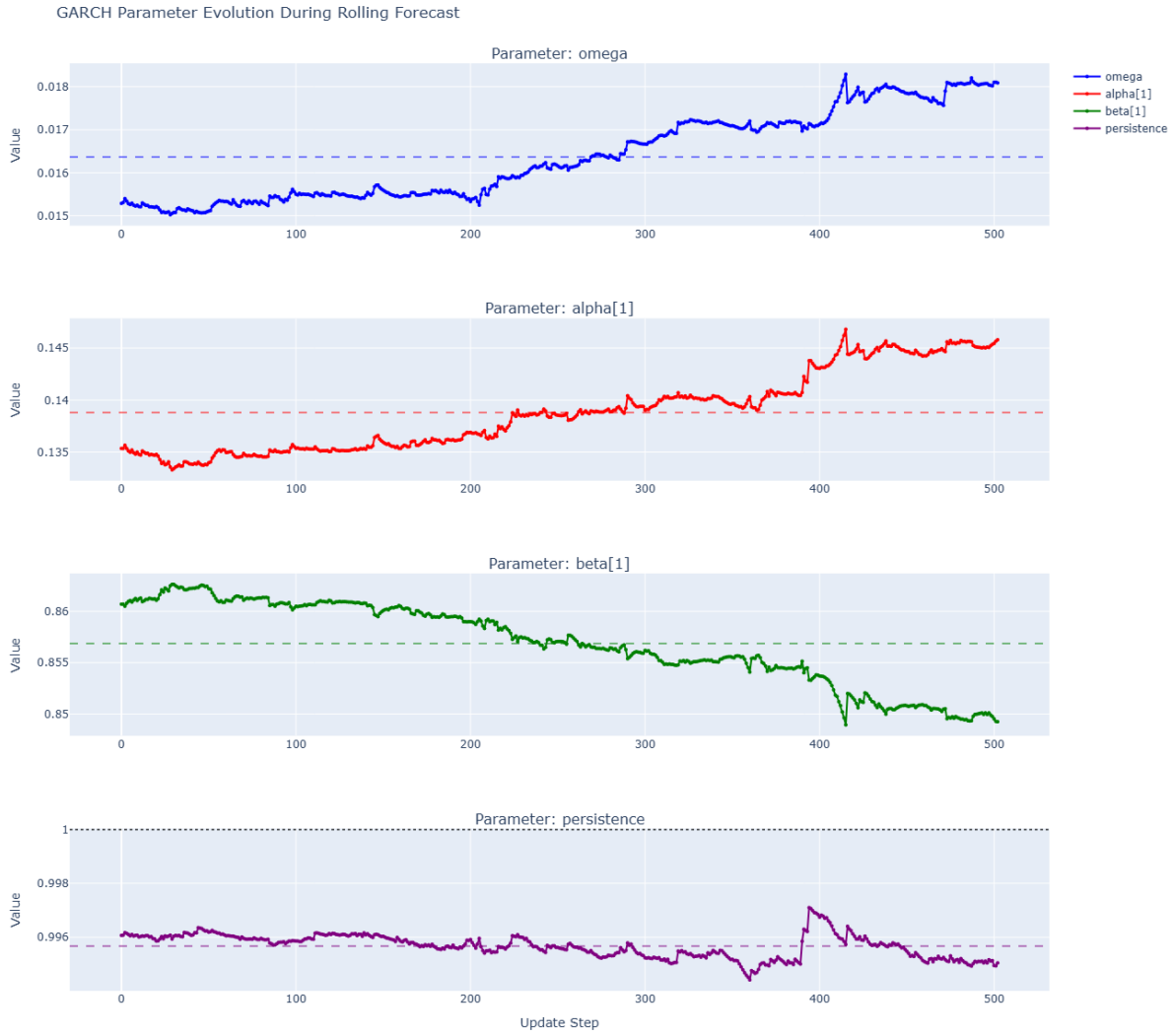

Figure 15: Parameter evolution over time

4.3.1 Comparison with VIX

The VIX is an index that measures the expected volatility of the US market over the next 30 days. Higher VIX, higher the volatility is expected to be. It is calculated from option prices of the SP500 (i.e. if investors pay more for the options, more volatility is expected).

Since our GARCH also measures volatility, let's compare them:

They look very very similar (Figure 18)! This means that our GARCH(1,1) model is effectively capturing the market's expectation of future volatility as represented by the VIX index.

Let's now look at distribution of the difference between our estimated volatility and VIX's:

Table 5: Summary Statistics

Statistic	**Value**
Mean	2.90
Std. Dev.	4.65
Min	-37.81
25%	1.03
50% (Median)	2.86
75%	5.10
Max	28.09

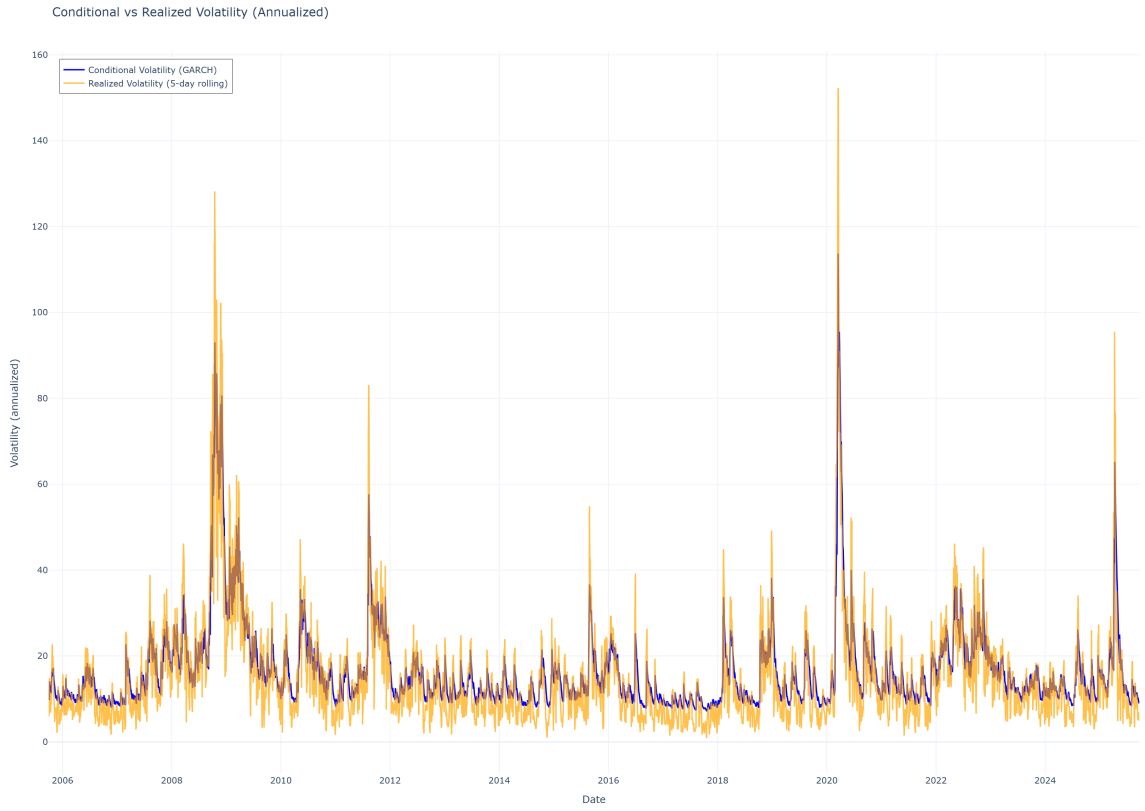

Figure 16: Daily trained vs Trained Once conditional volatilities

Looking at the distribution (Figure 19) of the errors and the graph above, we can interpret as follows: The long left tail indicates that we sometimes overestimate the VIX significantly, while the shorter right tail suggests that our underestimations are generally smaller in magnitude, but much more frequent! If we look at Figure 18, we can see that the conditional volatility is almost always below the vix (mostly in periods of calm), but we go over the index on large spikes (which are less frequent but make our model overreact, plus the high persistence of our model makes it stay high for a while).

5 Regime Classifiers

The GARCH model spits out many volatility values. However, the general public doesn't understand the difference between a squared variance of 0.04 and 0.05. They just want to know if the market is in a "high volatility", "medium" or "low volatility" period.

These abstract levels of volatility are called Regimes, and identifying which regime we were into on past (or even current) periods is called Regime Classifying. On this pratice we will use two methods: Hidden Markov Models and Markov Switching Autoregressive Model, both linked to Markov Chains.

5.1 Hidden Markov Model

To understand HMM, we first need to understand Markov Chains.

A Markov Chain is a system that transitions from one state to another, where the probability of moving to the next state depends only on the current state, not on the sequence of events that preceded it (i.e. we only care about today to tell what we'll do tomorrow). That is, it has no memory of the past (only today matters).

So for every state, we have a probability of going to any other state (including itself). This probabilities can be stored in a matrix called the transition matrix.

$$P = [P_{ij}], \quad \text{where } P_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$$

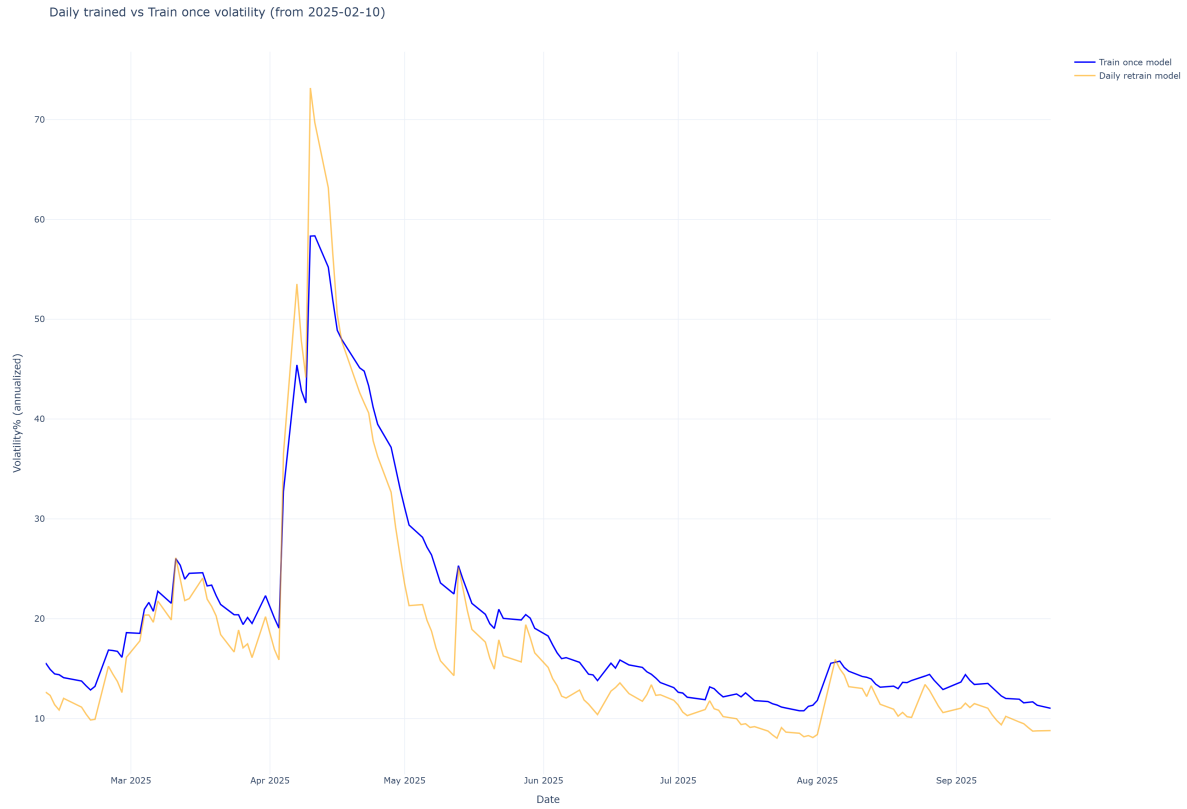

Figure 17: Zoomed Daily Trained model vs Trained Once model

And the memory-less property can be expressed as:

$$P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \ldots, X_0) = P(X_{t+1} = s_j \mid X_t = s_i)$$

That is, the probability of transitioning to another state depends only on the current state (X_t) and not on any previous states ($X_{t-1}, \ldots, X_0$). So we don't mind if we lack some farther past value.

The classical example is as follows:

Suppose we have two states:

- s_1 = Sunny
- s_2 = Rainy

then the transition matrix is:

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

- The rows correspond to the current state (today).
- The columns correspond to the next state (tomorrow).
- Each row sums to 1, because they represent probabilities of all possible next states.

So once we have the matrix, we can say that:

- if today is Sunny (s_1):
  - probability of tomorrow being Sunny = 0.8

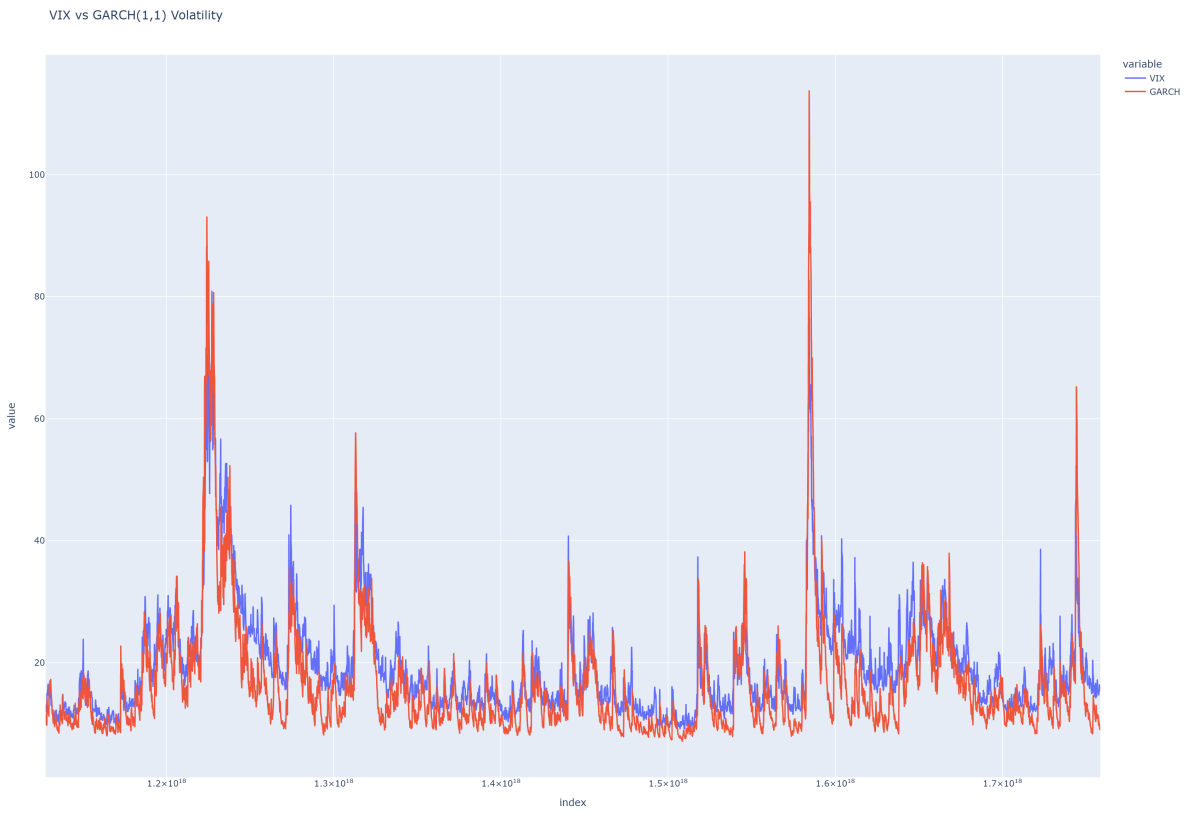

Figure 18: Vix vs GARCH volatility

  - probability of tomorrow being Rainy = 0.2

- if today is Rainy (s_2):
  - probability of tomorrow being Sunny = 0.4
  - probability of tomorrow being Rainy = 0.6

We can start seeing what is the connection to Hidden Markov models (which is a model that assumes the memoryless property of Markov chains). On the HMM, the states are hidden (always). Continuing with the classic example, we don't know if today is sunny or rainy, but we have some observable data that depends on the state (e.g., if people are carrying umbrellas, it is more likely to be rainy).

But the "we only care about the present" filosophy is maintained. That is, the hidden states behave as a markov chain.

For a HMM, we have the following components:

1. States set: $S = \{s_1, s_2, \ldots, s_N\}$ (hidden states)
2. Observations set: $O = \{o_1, o_2, \ldots, o_M\}$ (observable outputs)
3. Transition probabilities: $A = [a_{ij}]$ where $a_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$
4. Emission probabilities: $B = [b_{jk}]$ where $b_{jk} = P(Y_t = o_k \mid X_t = s_j)$

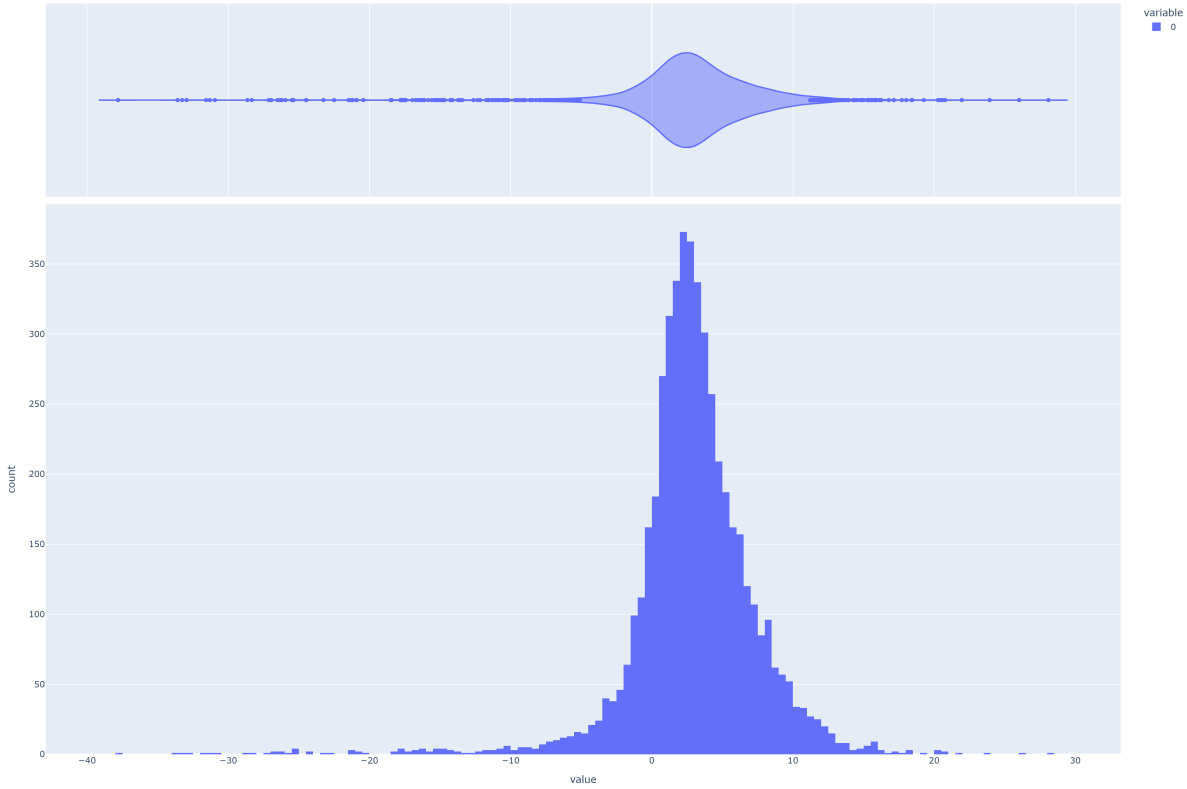

Figure 19: VIX - GARCH distribution

5. Initial state distribution: $\pi = [\pi_i]$ where
   $\pi_i = P(X_1 = s_i)$

Let's ground it with the rainy/sunny example: - States: $S = \{s_1 = \text{Sunny}, s_2 = \text{Rainy}\}$ - Observations: $O = \{o_1 = \text{No Umbrella}, o_2 = \text{Umbrella}\}$ - Transition matrix:

$$A = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

- Emission matrix:

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$$

The emission matrix tells us the probability of observing an umbrella or not, given the weather state. For example, if it's sunny, there's a 90% chance of seeing no umbrella, and if it's rainy, there's a 80% chance of seeing an umbrella, and a 20% chance of seeing no umbrella.

- Initial distribution: $\pi = [0.6, 0.4]$ (60% chance of starting sunny, 40% rainy)

In our use case, the umbrellas (observations) are the volatility values we gt by doing the filtering / or 1day forecast wth GARCH, and the hidden states are the volatility regimes (e.g., high volatility, medium, low volatility, ...).

So now let's apply the theory to our volatility data! We are going to use 3 hidden states (i.e. low medium and high vol).

But how does the HMM work? Basically the model is initialized with random values for the means, variances, transition probabilities, etc... Then, it uses the Baum-Welch algorithm (that combines two other algorithms until convergence or hitting the limit: forward and backward) to iteratively adjust these parameters to maximize the likelihood of the observed data (i.e. the volatilities we got from GARCH).

After fitting it, the model obtains through this some n_states clusters with means and variances, and also the transition matrix.

```
State means (mus): [10.56188403 17.07205403 34.91428278]
State standard deviations (sigmas): [ 1.43257215  2.91549997 15.72113009]
Transition matrix (transmat):
 [[9.74158095e-01 2.45522167e-02 1.28968871e-03]
 [3.10906763e-02 9.58135226e-01 1.07740981e-02]
 [1.13255006e-54 3.45442454e-02 9.65455755e-01]]
```

This data tells us that:

- **State means:**
  - State 1: $\mu \approx 10.56$ (low volatility regime)
  - State 2: $\mu \approx 17.07$ (medium volatility regime)
  - State 3: $\mu \approx 34.91$ (high volatility regime)
- **State standard deviations:**
  - State 1: $\sigma \approx 1.43$ (very stable)
  - State 2: $\sigma \approx 2.92$ (moderate variability)
  - State 3: $\sigma \approx 15.72$ (high instability)
- **Transition matrix (P):**

$$P = \begin{bmatrix} 0.9742 & 0.0246 & 0.0013 \\ 0.0311 & 0.9581 & 0.0108 \\ 0.0000 & 0.0345 & 0.9655 \end{bmatrix}$$

  - High persistence in all regimes (the diagonal entries are very close to 1).
  - Low $\leftrightarrow$ High is almost impossible.

So now let's plot it:

It is a really weird-looking graph (Figure 20), but what does it mean exactly? The plot above is made up of columns (thin, but columns anyway). Each column has a colour, or several. When a column has a single color, it means that the probability of that state being the regime is 100%. If it has two colors, it means that the mode is unsure, therefore it might assign 70% to one color (one regime) and 30% to another one. The same happens when on a single column there are three colours.

NOTE: It doesn't matter which color the GARCH line touches.

This plot is the equivalent of writing that for each column (i.e. each date), there are probabilities [Pr(low vol), Pr(mid vol), Pr(high vol)]

Each column represents a row of the observation matrix, for each value of GARCH at a point in time. It is hard to see, but each column can contain the three probabilities of the states. However, the fact that it is hard to see tells us that the model switches with a lot of confidence between states (i.e. there is no period of time in which the probability of being on one of the states is not over 0.8).

We can also see that the model makes clusters of regimes (we can observe this with the 2008 financial crisis, the 2011 European debt crisis, the 2020 COVID-19 crisis, and the 2022 inflation crisis). That is, the model doesnt just indicate that there is high volatility on one day where the conditional volatility spikes, and then it returns to normality. It waits until the volatility settles to change the regime.

So we have effectively developed a model that can tell us if we were on a high, medium or low volatility regime, based on the GARCH volatility values!

5.2 Markov Switching Autoregression Model

However, we also see with the previous plot that the model is very sure when it changes the regime. However, in real life, this is not the case. We don't wake up one morning and say: High voaltility is over! Now we are in low volatility! Instead, we transition slowly between regimes.

To account for this property, we use the Markov Switching Autoregression Model (MSAR). This model is very similar to the HMM, but it forces the transition matrix to have high values on the diagonal

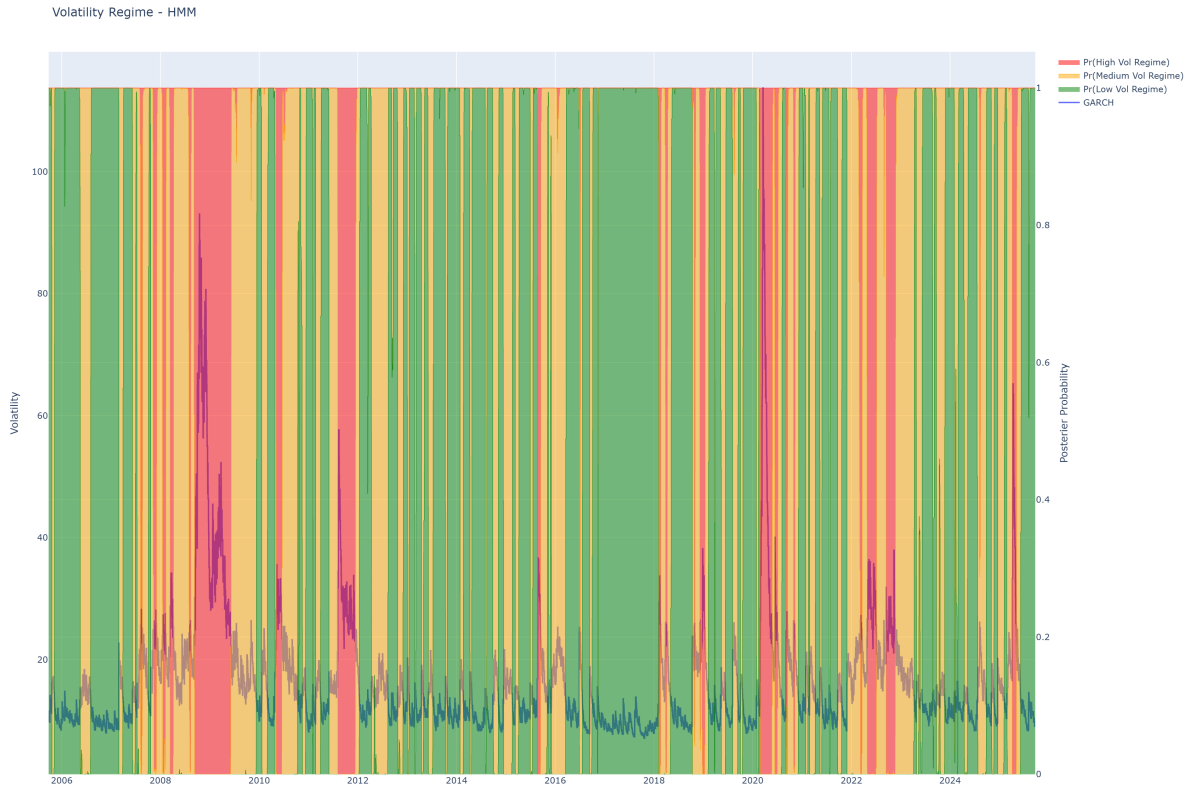

Figure 20: HMM Volatility Regimes and GARCH

(i.e. high probability of staying in the same state). This way, we force the model to be more "sticky" with the regimes. Note that the memoryless Markov property is still applied (hence the name), but we are also making past values influence the model (that's why it's called autoregressive!)

It defines properties specifically for each regime (for ex. in low volatility regime, returns follow an AR(1) with small σ^2, and on high vol, returns follow an AR(1) with a higher σ^2 and possibly different mean reversion)

And exactly as expected, here we can see that the model sticks more to the states (i.e. there are more "curves", which mean that the probabilities of being in a state change more slowly).

5.3 Comparison

A useful method for comparing how well each model fits the data is the log-likelihood. The log-likelihood measures how probable the observed data is, given the model parameters. A higher log-likelihood indicates a better fit to the data.

```
log-likelihood of HMM: -12596.045936794613
Transition Matrix of HMM:
[[9.74158095e-01 2.45522167e-02 1.28968871e-03]
 [3.10906763e-02 9.58135226e-01 1.07740981e-02]
 [1.13255006e-54 3.45442454e-02 9.65455755e-01]]

Log-likelihood of MSAR: 16446.800669230575
Transition Matrix of MSAR:
[[9.70666336e-01] [3.33071545e-02] [2.11475002e-06]
[2.88565606e-02] [9.60615885e-01] [3.83404736e-02]
[4.77103580e-04] [6.07696087e-03] [9.61657412e-01]]
```

We see that both models say that states are very persistent (over 95% of staying). The higher log-likelihood value of the MSAR indicates that it is a better fit for the input data (i.e. the GARCH

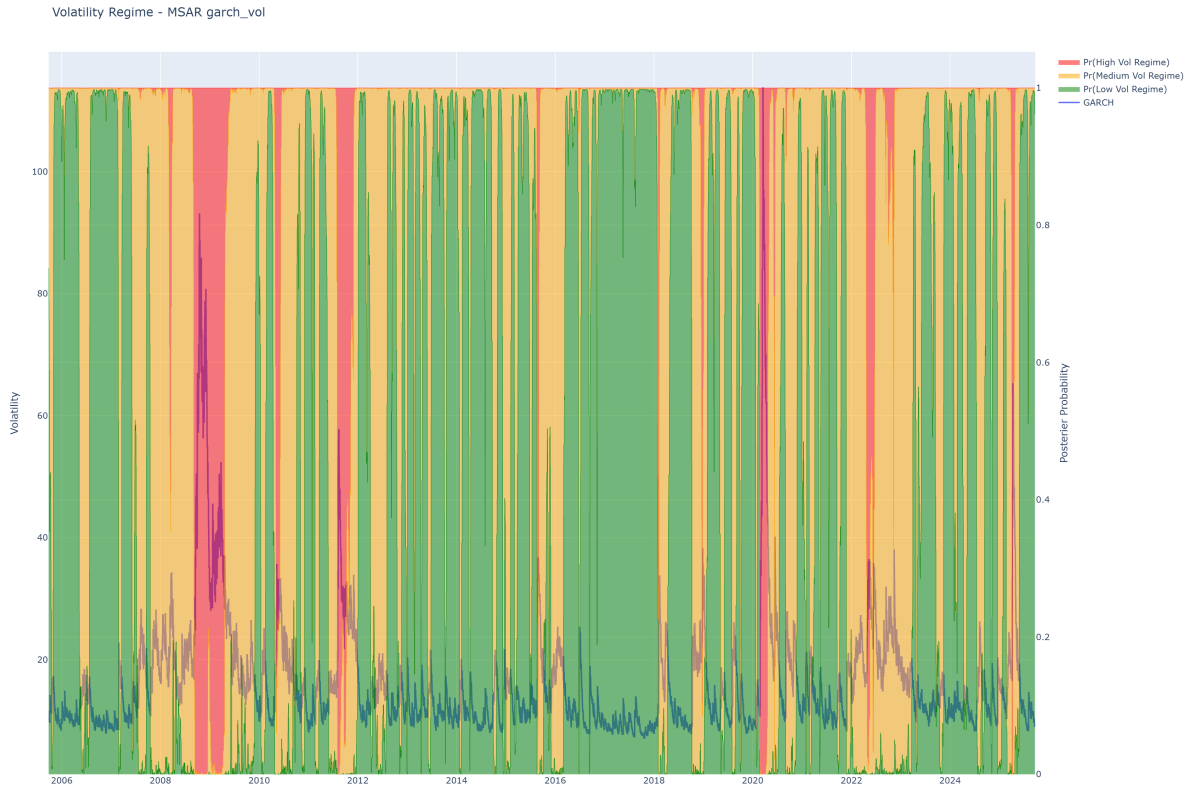

Figure 21: MSAR Regime Classifier GARCH

volatilities). This is expected, since the MSAR model accounts for the persistence of volatility regimes, which is a known characteristic of financial markets.

However, we can see that the MSAR model is not necessarily more "sticky" than the HMM. In fact, the HMM has a higher probability of staying in the low volatility regime and the high volatility regime. The MSAR model has a higher probability of staying in the medium volatility regime. This means that the MSAR model is more likely to switch between low and high volatility regimes, while the HMM model is more likely to stay in the same regime.

6 Conclusion

This practice has been much more than just an exercise in applying volatility models. It has been a way for me to truly understand how modern methods like GARCH and regime-switching models work in practice, and what they can (and cannot) tell us about financial markets.

Paradoxically, I intended to make it not very rigorous (I wanted to focus on understanding the topic, rather than proving everything around it), but I ended up discovering many statistical tests that help check the assumptions behind these models and to choose between them.

The real value, though, has gone beyond the models themselves. Working through the material has made me more aware of the assumptions hidden in the simplest things we often take for granted, like how we define returns, or what we assume about prices and distributions. It has also shown me how research in finance differs from pure mathematics: while math often focuses on very intrincate theorems, finance requires constant confrontation with messy, imperfect data.

Along the way, I've revisited key probability, explored hypothesis testing, and discovered new areas of quantitative finance. I've also seen how GARCH models can be useful in real applications, while at the same time realizing their limitations. Most importantly, I've learned the value of persistence: to keep digging until I find an explanation that makes sense to me, even when the literature feels too dense or overly technical.

In the end, this notebook has been both a learning tool and a personal exploration by helping me

connect theory with practice, and motivating me to keep asking questions that lead to deeper understanding.

Disclaimer on the use of AI:
I have used AI assistants (ChatGPT) for coding and formatting and formulas.

---

7 References

- Maximum Likelihood in the GJR-GARCH(1,1) Model (CrossValidated)
- Maximum Likelihood Estimation (Bookdown)
- Markov Chains (Brilliant.org)
- Markov Chains (YouTube video)
- ARCH Definition (Investopedia)
- Market Regime Detection Using HMMs (QuantStart)
- SSRN Paper on Volatility Modelling
- ARCH Test Explained (NumXL)
- Understanding VIX with GARCH and ARIMA (Medium)
- ARCH Model (FSB Miami)
- arch.univariate.HARX Documentation
- GARCH Forecasting under Different Distribution Assumptions (Willhelmesson, 2006)
- Gregory Gundersen – Log Returns
- Returns vs Log Returns (Quant.SE)
- Predicting Volatility with HAR Models (SR-SV Blog)
- Practical Time Series Analysis – Code Repo (GitHub)
- HMMLearn Documentation
- ECB Volatility Regime (2018)
- ARCH and Regime Changes (ScienceDirect)
- Kim, Nelson, Startz (1998) – Three-State Variance Switching
- Statsmodels – Markov Autoregression Example
- Statsmodels – Markov Regression Example